

Browser API agent

Working plan

Status: interactive planning; no implementation or authenticated site exploration yet.

Goal and agreed decisions

Build a reusable engine that accepts an API specification, learns a website through a legitimate user's browser session, and executes supported endpoint requests through that site's UI. Powder is the first test site.

- Browser operations may use page content, DOM/accessibility, forms, uploads, and downloads.
- Business operations must go through the UI. Replaying authenticated network requests is outside the agreed scope.
- The first prototype runs locally with a dedicated browser profile.
- Target exact behavior for a declared supported endpoint subset, with explicit coverage gaps.
- A dedicated test account will be available to exercise writes and jobs on test data during the implementation and validation phase.
- Human participation establishes or renews login. Proposed implementation: the user signs in directly in the controlled browser; credentials never enter prompts or learned site memory.
- This stage is planning, not building or operating the target account.

Proposed architecture

1. Contract intake and coverage

Parse routes, methods, inputs, response schemas, status codes, identifiers, pagination, and job semantics from the supplied specification. Track each endpoint as unexamined, mapped, verified, constrained, or unsupported. A valid response shape alone is insufficient to establish compatibility.

2. Discovery and durable knowledge

Explore the main navigation to learn the application's entities and relationships, then investigate the workflows needed by the target endpoints. Do not require an exhaustive crawl before supporting the first endpoint.

Persist separately:

- A site map: page states, navigation paths, entity relationships, forms, controls, and relevant permissions.
- Endpoint recipes: input bindings, semantic locators, prerequisites, steps, waits, extraction rules, postconditions, and retry policy.
- Account-scoped state: entity identifiers and UI locations, operation receipts, job tracking, and session references.

Version recipes and record their verification evidence and applicable site/account configuration. Remember how to retrieve live data; do not silently serve stale remembered business values. Keep credentials and session secrets separate from reusable knowledge.

3. Recipe verification

Verify complete outcomes using authorized test data and settings. Check submitted settings, entity identity, resulting state, response values, and contract conformance. Learning where a submit button is does not establish that a workflow is safe to replay or that its result matches the endpoint.

Use the dedicated test account for this phase. Select concrete test documents and mutation boundaries before beginning authenticated validation; the current session remains planning only.

4. Fast execution

Validate request → resolve account and identifiers → select a verified recipe → acquire browser ownership → execute UI steps with lightweight assertions → verify outcome → extract and validate the API response.

Use deterministic browser actions for established recipes. Invoke the reasoning agent for uncovered operations, unexpected states, or bounded recovery. Prefer semantic locators, observed deep links, event-based waits, warm sessions, and complete table/download extraction where available. Measure speed; do not promise API-like latency before benchmarking the site.

5. Recovery and durable operations

Keep a durable operation record before submitting a mutation or job. On timeout, inspect whether submission succeeded before retrying. An uncertain outcome must not trigger blind replay. Reconcile using stable identifiers and available UI evidence; otherwise request human assistance through a separate control channel.

Checkpoint jobs so monitoring can resume after a worker restart. Match the target API's submission and polling contract; do not introduce a different asynchronous response format without explicitly declaring the compatibility change. Reauthentication pauses browser work and returns browser control to the user.

Proposed first milestone

Powder's overview documents:

Endpoint	Documented purpose	UI mapping to investigate
POST /file_uploads	Upload a document, optionally associated with a portfolio	Upload form, settings, portfolio selection, resulting upload identity
GET /file_uploads/{id}	Retrieve processing status	Upload detail/status view and its state transitions
GET /file_uploads/{id}/data	Retrieve processed structured data	Result views, pagination, and available exports

Source: [Powder API overview](#).

The overview also mentions /configure_webhook. UI feasibility is unverified. General CRUD support is not established by this page. The full endpoint schemas still need to be obtained and the authenticated UI has not been inspected. No exact field mappings or full API compatibility are established yet.

Proposed proof: complete upload → status → results for a known test document, then repeat with varied inputs and relevant failures. Test expired login, changed page structure, and interrupted submission. Measure correctness, warm-request latency, recovery frequency, and duplicate submissions; measure site processing time separately from adapter overhead.

Open decisions

1. Confirm the proposed initial endpoint set and obtain its complete contract and the Powder application URL.
2. Select concrete test data and limits for discovery writes and jobs.
3. Define expected concurrency and latency targets.
4. Review the proposed implementation choices below.

Proposed implementation starting point

- TypeScript service with a local HTTP interface derived from the supplied API contract.
- Playwright for browser control, with semantic locators and built-in actionability waits. See [locators](#) and [auto-waiting](#).
- A constrained workflow format interpreted by the browser runner, so recipes can be inspected, versioned, validated, and repaired.
- SQLite for the local site map, workflow metadata, account-scoped identifiers, and durable operation state; versioned recipe files for review.
- A reasoning-model interface used for discovery and recovery. Model/vendor selection remains open.
- A separate local control interface for human login, coverage review, and operations requiring attention. Preserve the supplied endpoint's response contract on the API interface.
- Initially serialize browser interactions for one test account. Interleave short job-status checks with other work rather than holding browser ownership while a remote job processes.

These technology choices and concurrency defaults are proposals, not user decisions.

Next planning discussion: review discovery and recipe memory, then settle the first milestone's contract, test data, and measurable acceptance criteria before implementation.